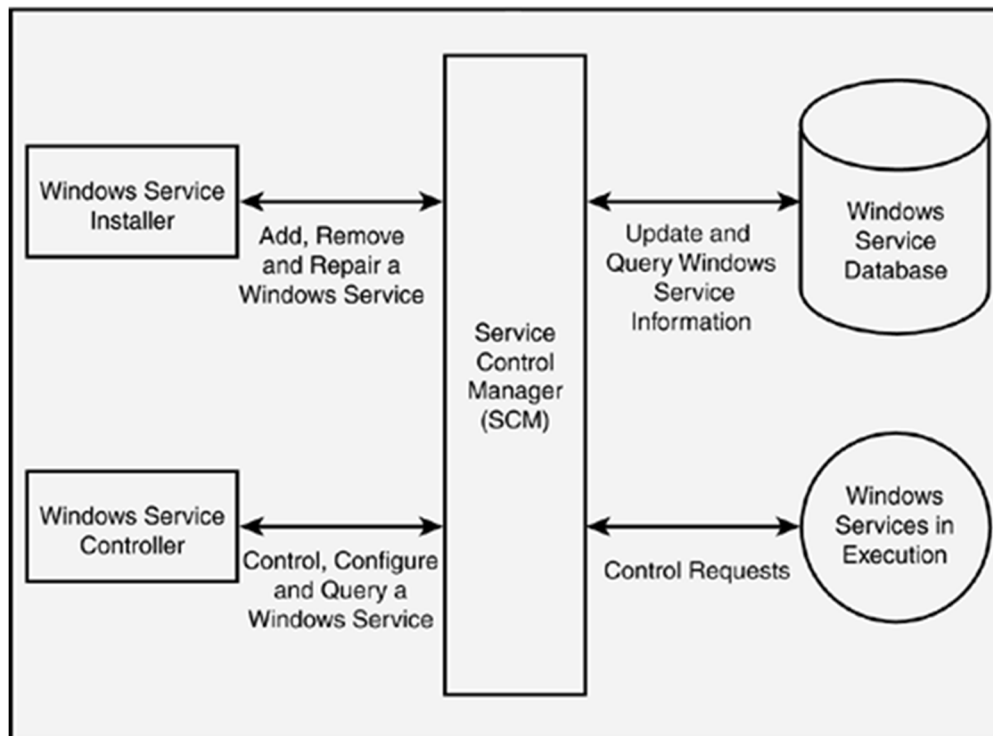


Servicii Windows și sistemul de logging

- Serviciile sunt aplicații executabile ce se execută permanent (sau de lungă durată) în fundal, în propria sesiune Windows.
- Serviciile sunt automat pornite, uzual înainte de crearea unei sesiuni utilizator (înainte de a se conecta un utilizator în sistem)
- Serviciile pot fi oprite, puse pe pauză sau repornite, fără a avea o interfață proprie cu utilizatorul
- Interfața sistem cu utilizatorul pentru vizualizarea, configurarea, pornirea și oprirea serviciilor se face prin Services Control Manager
- Serviciile se pot utiliza pentru:
 - Operații de fundal de lungă durată
 - Aplicații de tip server (ce deservește diferite aplicații client în rețea, baze de date etc.)
 - Operații de monitorizare ale sistemului sau activității
- Serviciile pot folosi și un context de execuție al unui anumit utilizator din sistem, nu neapărat cel login-at la un moment dat

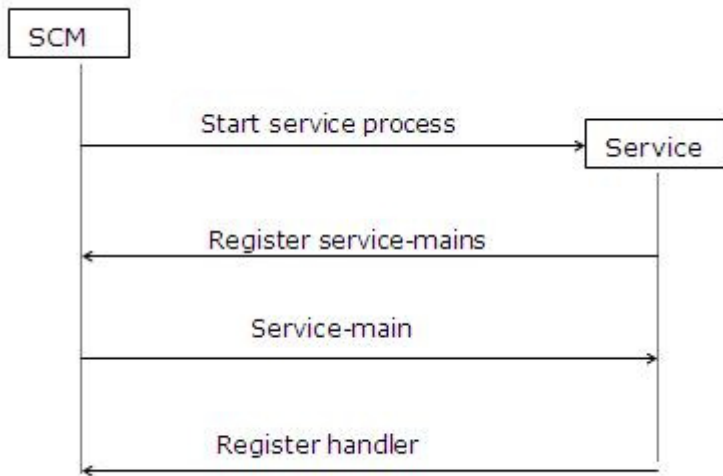
Arhitectura sistemului de servicii



Funcționarea și implementarea serviciilor

- Serviciile sunt aplicații ce respectă un anumit protocol de comunicare cu SCM

- SCM: start service
- Serviciul: își înregistrează funcția principală a serviciului la SCM
- SCM: pornește funcția main a serviciului
- Serviciul: își înregistrează funcția de tratare a comenzilor de la SCM



- Pornirea unui serviciu

```

//
// Purpose:
//   Entry point for the process
//
// Parameters:
//   None
//
// Return value:
//   None, defaults to 0 (zero)
//
int __cdecl _tmain(int argc, TCHAR *argv[])
{
    // If command-line parameter is "install", install the service.
    // Otherwise, the service is probably being started by the SCM.

    if( lstrcmpi( argv[1], TEXT("install")) == 0 )
    {
        SvcInstall();
        return;
    }

    // TO_DO: Add any additional services for the process to this table.
    SERVICE_TABLE_ENTRY DispatchTable[] =
    {
        { SVCNAME, (LPSERVICE_MAIN_FUNCTION) SvcMain },
        { NULL, NULL }
    };

    // This call returns when the service has stopped.
    // The process should simply terminate when the call returns.

    if (!StartServiceCtrlDispatcher( DispatchTable ))
    {
        SvcReportEvent(TEXT("StartServiceCtrlDispatcher"));
    }
}

```

- SERVICE_TABLE_ENTRY

```

typedef struct _SERVICE_TABLE_ENTRYW {
    LPWSTR          lpServiceName;
    LPSERVICE_MAIN_FUNCTIONW lpServiceProc;
} SERVICE_TABLE_ENTRYW, *LPSERVICE_TABLE_ENTRYW;

```

- SERVICE_MAIN_FUNCTION

```
LPSERVICE_MAIN_FUNCTIONW LpServiceMainFunctionw;
```

```
void LpServiceMainFunctionw(  
    [in] DWORD dwNumServicesArgs,  
    [in] LPWSTR *lpServiceArgVectors  
)  
{...}
```

- StartServiceCtrlDispatcher

```
BOOL StartServiceCtrlDispatcherW(  
    [in] const SERVICE_TABLE_ENTRYW *lpServiceStartTable  
);
```

- Service main

```

//
// Purpose:
//   Entry point for the service
//
// Parameters:
//   dwArgc - Number of arguments in the lpszArgv array
//   lpszArgv - Array of strings. The first string is the name of
//             the service and subsequent strings are passed by the process
//             that called the StartService function to start the service.
//
// Return value:
//   None.
//
VOID WINAPI SvcMain( DWORD dwArgc, LPTSTR *lpszArgv )
{
    // Register the handler function for the service

    gSvcStatusHandle = RegisterServiceCtrlHandler(
        SVCNAME,
        SvcCtrlHandler);

    if( !gSvcStatusHandle )
    {
        SvcReportEvent(TEXT("RegisterServiceCtrlHandler"));
        return;
    }

    // These SERVICE_STATUS members remain as set here

    gSvcStatus.dwServiceType = SERVICE_WIN32_OWN_PROCESS;
    gSvcStatus.dwServiceSpecificExitCode = 0;

    // Report initial status to the SCM

    ReportSvcStatus( SERVICE_START_PENDING, NO_ERROR, 3000 );

    // Perform service-specific initialization and work.

    SvcInit( dwArgc, lpszArgv );
}

```

- RegisterServiceCtrlHandler

```

SERVICE_STATUS_HANDLE RegisterServiceCtrlHandlerW(
    [in] LPCWSTR          lpServiceName,
    [in] LPHANDLER_FUNCTION lpHandlerProc
);

```

- SetServiceStatus
 - SERVICE_START_PENDING
 - SERVICE_RUNNING
 - SERVICE_STOPPED

```
BOOL SetServiceStatus(  
    [in] SERVICE_STATUS_HANDLE hServiceStatus,  
    [in] LPSERVICE_STATUS      lpServiceStatus  
);
```

- Service control handler

```

//
// Purpose:
//   Called by SCM whenever a control code is sent to the service
//   using the ControlService function.
//
// Parameters:
//   dwCtrl - control code
//
// Return value:
//   None
//
VOID WINAPI SvcCtrlHandler( DWORD dwCtrl )
{
    // Handle the requested control code.

    switch(dwCtrl)
    {
        case SERVICE_CONTROL_STOP:
            ReportSvcStatus(SERVICE_STOP_PENDING, NO_ERROR, 0);

            // Signal the service to stop.

            SetEvent(ghSvcStopEvent);
            ReportSvcStatus(gSvcStatus.dwCurrentState, NO_ERROR, 0);

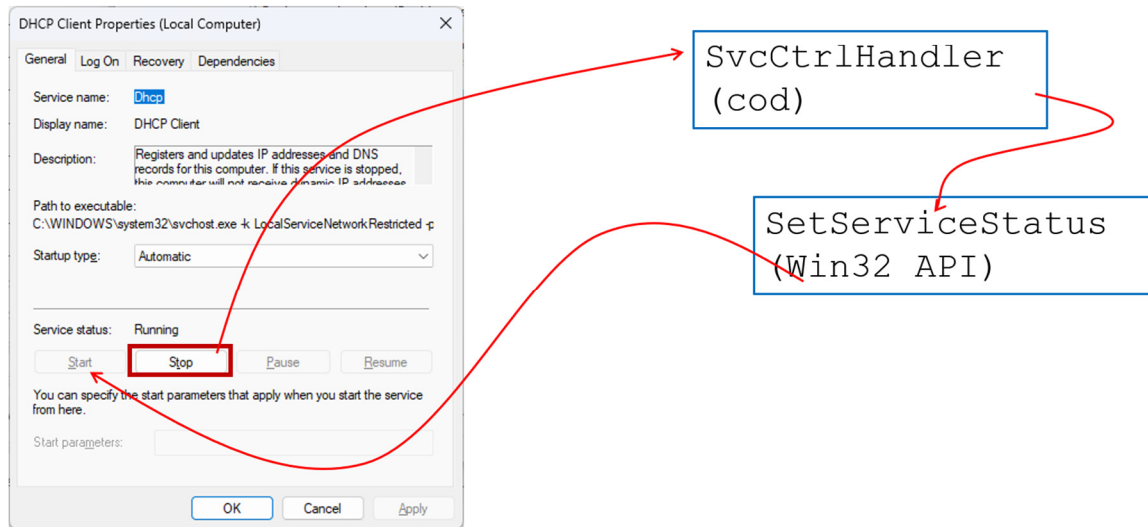
            return;

        case SERVICE_CONTROL_INTERROGATE:
            break;

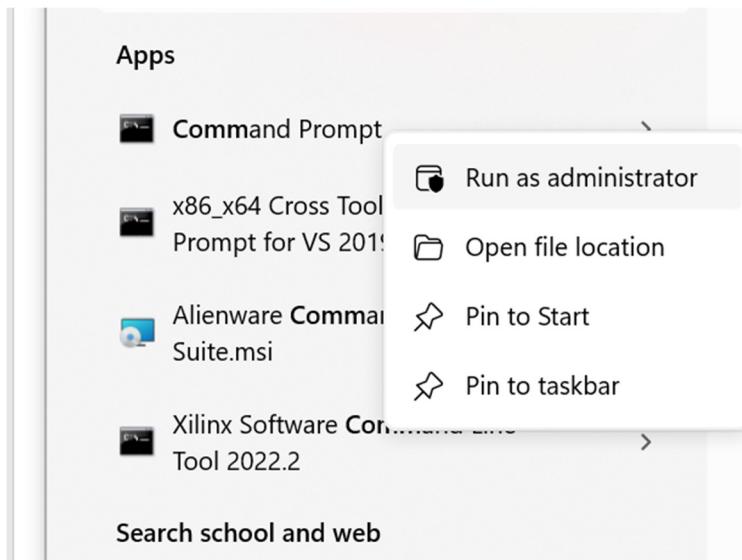
        default:
            break;
    }
}

```

Servicii sistem



Pornirea unei app in modul administrator



SCM – Service Control Manager

sc query dhcp

sc start nume_serviciu

sc stop nume_serviciu

sc pause nume_serviciu

sc create nume_serviciu

sc delete nume_serviciu